

Guide de Migration : Assignations et États vers Flowable BPMN

Pour reproduire les assignations de la version 1 dans Flowable, vous n'avez plus besoin d'écrire ces règles en dur dans vos `Services`. Tout se configure directement dans le modèle XML (ou via l'interface graphique **Flowable Modeler**).

Ce document explique comment modéliser les concepts de votre V1 (sans Flowable) vers la V2 (avec Flowable).

1. Assigner une tâche à un rôle (Remplace le `roleRequired`)

Dans la V1, vous vérifiiez si la personne avait le bon rôle pour passer à l'état suivant (via `workflowService.validateTransitionForIncident()`). Dans Flowable, on utilise les **Groupes Candidats** (*Candidate Groups*). Toute personne ayant ce rôle verra la tâche dans sa liste d'attente.

- **Dans le Flowable Modeler (Interface)** : Sélectionnez votre `User Task` -> allez dans les propriétés à droite -> cherchez **Assignments** -> remplissez le champ **Candidate groups** (ex: `Opérateur_médical`).
- **Dans le XML BPMN** : Flowable ajoute l'attribut `flowable:candidateGroups`.

```
<userTask id="taskEnCours" name="En cours"
flowable:candidateGroups="Opérateur_médical">
</userTask>
```

2. Assigner à un utilisateur spécifique (Remplace `incident.setAssignedTo()`)

Si vous voulez qu'une tâche soit assignée à un utilisateur précis (soit un ID fixe, soit une variable dynamique), on utilise l'**Assignataire** (*Assignee*).

- **Dans le Flowable Modeler (Interface)** : Sélectionnez la tâche -> **Assignments** -> remplissez le champ **Assignee**. Vous pouvez utiliser une variable dynamique comme `${technicienId}` (que vous passez au démarrage du processus).
- **Dans le XML BPMN** : Flowable ajoute l'attribut `flowable:assignee`.

```
<userTask id="taskAssigne" name="Assigné" flowable:assignee="${technicienId}">
</userTask>
```

3. Logique d'assignation complexe (Remplace les règles métiers `if/else`)

Dans votre V1, vous aviez une règle : "Si la catégorie est Médical, trouver le premier opérateur médical et lui assigner". Pour faire cela proprement dans Flowable, on attache un **Task Listener** (Écouteur de tâche) à la `userTask`.

Un listener exécute une classe Java personnalisée au moment où la tâche est créée.

- Dans le **Flowable Modeler (Interface)** : Sélectionnez la tâche -> **Task listeners** -> Ajoutez un écouteur sur l'événement **Create** et renseignez le chemin absolu de votre classe Java.
- Dans le **XML BPMN** :

```
<userTask id="taskNouveau" name="Nouveau">
  <extensionElements>
    <flowable:taskListener event="create"
class="com.netmar.incidentflow.listener.AssignationMedicaleListener" />
  </extensionElements>
</userTask>
```

Exemple de la classe Java associée (le listener) :

Au lieu de mettre le code dans le contrôleur ou le service (`IncidentService`), vous l'isolez dans une classe implémentant `TaskListener` :

```
package com.netmar.incidentflow.listener;

import org.flowable.engine.delegate.TaskListener;
import org.flowable.task.service.delegate.DelegateTask;
import org.springframework.stereotype.Component;

@Component
public class AssignationMedicaleListener implements TaskListener {
    @Override
    public void notify(DelegateTask delegateTask) {
        String categorie = (String) delegateTask.getVariable("category");
        if ("Médical".equalsIgnoreCase(categorie)) {
            // Logique pour trouver l'utilisateur en BDD...
            // userRepository.find...
            delegateTask.setAssignee("id_du_medecin");
        }
    }
}
```

Résumé de la migration

Concept V1 (Manuel)	Concept V2 (Flowable)
Changement d'état (setStatus)	Flèches de séquence (sequenceFlow)
Vérification des rôles (roleRequired)	Groupes candidats (candidateGroups)
Affectation utilisateur (setAssignedTo)	Assignataire (assignee)
Règles conditionnelles (if/else)	Écouteurs de tâches (taskListener)
Traçabilité (IncidentHistory)	Table d'audit automatique (act_hi_taskinst)